

MICROPROCESOARE SI LIMBAJE DE ASAMBLARE

Calculatoare 3 2025-2026

Curs 4 Sapt 4 21 octombrie 2025 16:00-18:00

serban@upit.ro

Modul de notare:

Examen:	50%	
Laborator:	30%	A2
Test (Midterm):	20%	A1

Planificarea activitatilor:

Laborator:

- saptamanal, conform orarului;
- sapt. 13, 14 refaceri laboratoare;

Curs:

- saptamanal, conform orarului;

Test (Midterm): in saptamana a 10-a (2 decembrie - 6 decembrie)

OBS. Este nevoie de nota minima 5 la toate activitatile (semestru – A1, A2 si sesiune – Examen!!! pentru promovare)

Continut

Introducere

Evolutia microprocesoarelor

Criterii de clasificare ale microprocesoarelor

Structura microprocesoarelor

Mecanisme specifice Microprocesoarelor

Microprocesoare pe 8 biti

Familia de microprocesoare x86

Familia de procesoare ARM

Bibliografie

The Design of a Microprocessor – Wilhelm G. Spruth – Springer

Microprocessor Design – Grant McFarland – McGraw-Hill – 2006

Microprocessor Interface Design – J.D. Nicoud – Chapman & Hall

Microprocessors and Microcomputers Hardware and Software - Ronald J. Tocci – Pearson

Intel Microprocessors – Barry Brey

Z-80 Microprocessor Architecture, Interfacing, Programming and Design - Ramesh Gaonkar

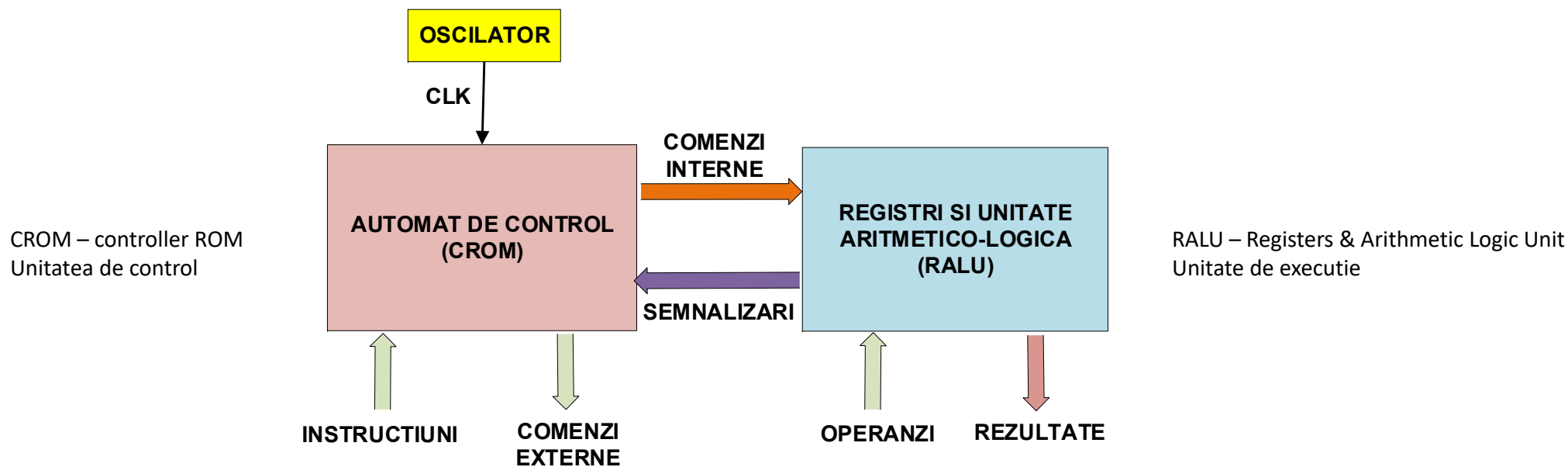
Introduction to 80X86 Assembly Language and Computer Architecture - Richard C. Detmer - Jones & Bartlett Pub (2001)

X86 Assembly Language and C Fundamentals – Joseph Cavanagh – CRC Press

ARM Assembly Language Fundamentals and Techniques, Second Edition - William Hohl

Fundamentals of Embedded Software Where C and Assembly Meet – Daniel Wesley Lewis

Structura unui microprocesor CISC

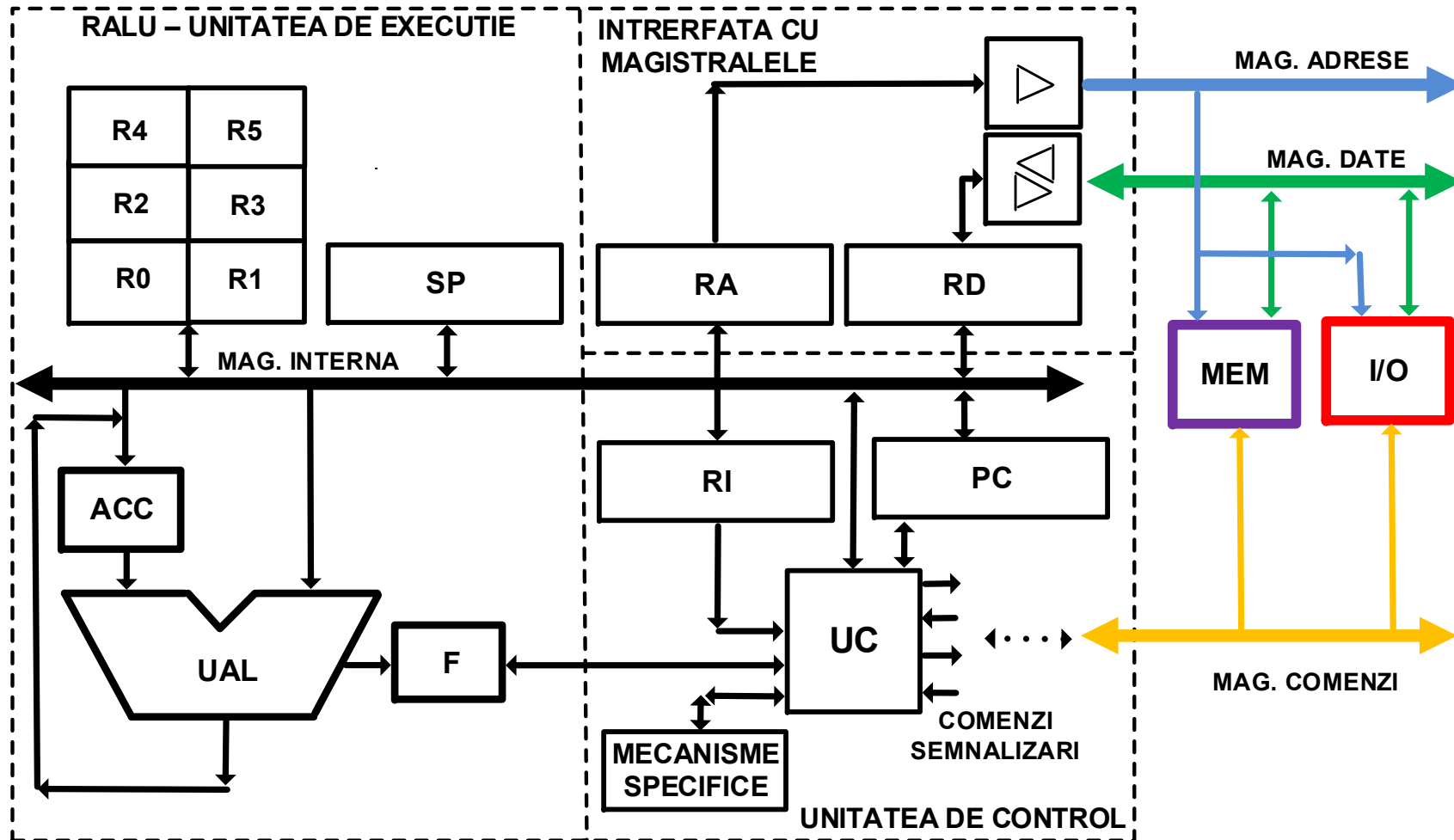


MAGISTRALA DE ADRESE (Address Bus) – Flux Comenzi Externe

MAGISTRALA DE DATE (Data Bus) – Fluxurile Instructiuni, Operanzi, Rezultate

MAGISTRALA DE CONTROL (Control Bus) – Flux Comenzi Externe

STRUCTURA UNUI PROCESOR VON NEUMANN



Notatiile care apar in structura sunt urmatoarele:

R0-R5 sunt registre generale de lucru, accesibile prin program.

SP (stack pointer) este registrul indicator al varfului stivei. Stiva este organizata in memorie, sensul de crestere a acesteia fiind catre adrese mici. Intotdeauna SP contine adresa de memorie la care s-a depus ultima valoare in stiva. Astfel, la o depunere in stiva, SP este decrementat si se inscrie valoare la adresa continuta de SP, iar la o extragere din stiva, se citeste cuvantul din memorie de la adresa data de SP, iar apoi SP este incrementat. Organizare date little endian (vs big endian).

RA este registrul de adrese, inaccesibil programatorului prin instructiuni. In registrul RA se incarca o adresa de memorie sau de port de intrare / iesire. Continutul RA este plasat pe magistrala externa de adrese in timpul operatiilor de citire sau scriere cu memoria sau cu un port de intrare / iesire. Conectarea registrului la magistrala externa se face prin intermediul unor circuite tampon, unidirectionale.

RD este registrul de date, de asemenea transparent programatorului. In acest registru este incarcat un cuvant (octet) de date citit sau scris in cadrul unei operatii cu memoria sau cu un port de intrare / iesire. Conectarea registrului la magistrala externa de date se face prin intermediul unor circuite tampon bidirectionale.

UAL reprezinta unitatea aritmetica logica care efectueaza toate operatiile aritmetice si logice.

ACC este registrul accumulator. La nivelul UAL registrul ACC se foloseste pentru pastrarea unui operand. Al doilea operand este furnizat direct de pe magistrala interna a procesorului. Rezultatul operatiei UAL se depune in ACC cu stergerea primului operand

F (flags) reprezinta registrul indicatorilor de conditie (bistabili), pozitionati in functie de rezultatele obtinute in UAL.

RI este registrul de instructiune, in care se incarca primul cuvant (octet) al instructiunii, codul instructiunii. Continutul registrului este analizat de unitatea de comanda, care recunoaste instructiunea si o executa.

UC este unitatea de comanda care citeste si decodifica instructiunile programului si, testand o serie de semnale de stare, lanseaza o succesiune de semnale de comanda care sa permita executia instructiunilor. UC-ul este de fapt un Automat microprogramat. Pentru fiecare instructiune preluata din memorie exista in automat un microprogram care o executa.

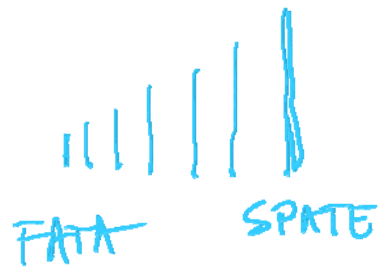
PC (program counter) este registrul numarator de program. La inceputul executiei instructiunii curente, PC contine adresa de memorie la care se gaseste primul cuvant al instructiunii (codul). In timpul executiei instructiunii curente continutul PC este actualizat, fie prin incrementare cu 1, 2 sau 3 (in functie de lungimea instructiunii curente) in cazul unei secvente liniare de instructiuni, fie prin incarcare cu o valoare citita sau calculata in cazul instructiunilor de ramificatie.

MEM este memoria sistemului de calcul.

I/O reprezinta dispozitivele de intrare / iesire impreuna cu interfetele lor.

In cadrul procesorului s-au demarcat prin linie intrerupta trei module importante: unitatea de executie, unitatea de interfata cu magistrala si unitatea de comanda.

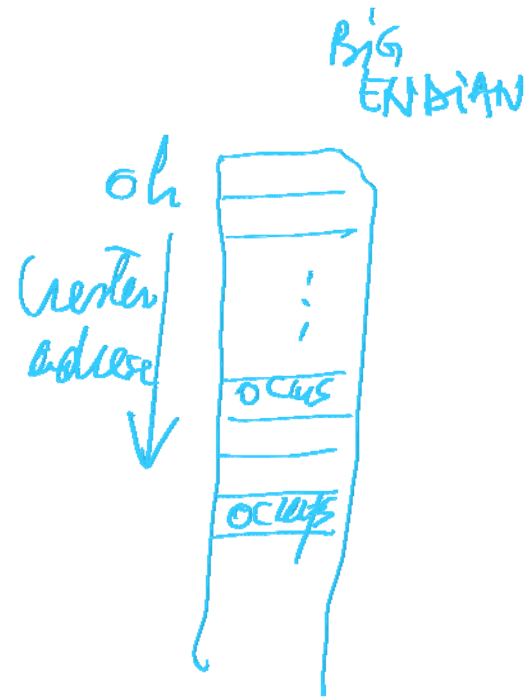
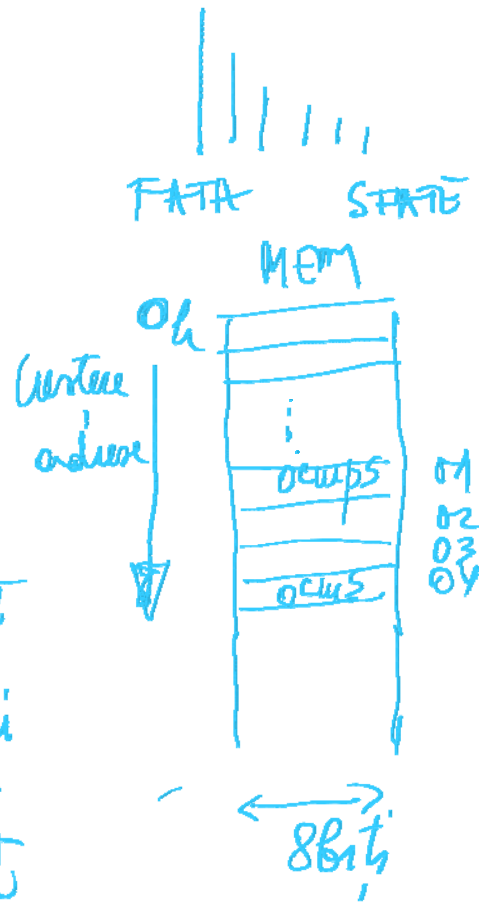
LITTLE ENDIAN VS BIG ENDIAN



4 octets

= OCHS - octet cell
(MSB) - most significant
most significant byte

= OCHPS - oct. cell
position significant
(LSB) - least significant
byte



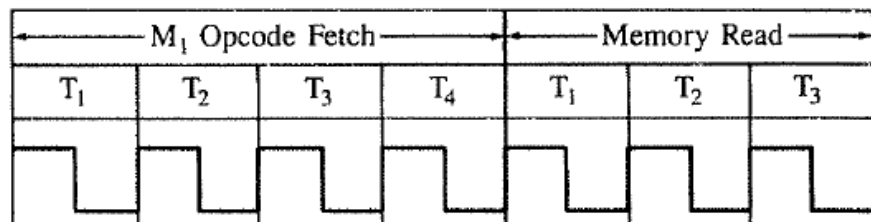
Executia instructiunilor

Instructiunile se executa de un procesor prin 1 sau mai multi cicli masina (functie de tipul instructiunii):

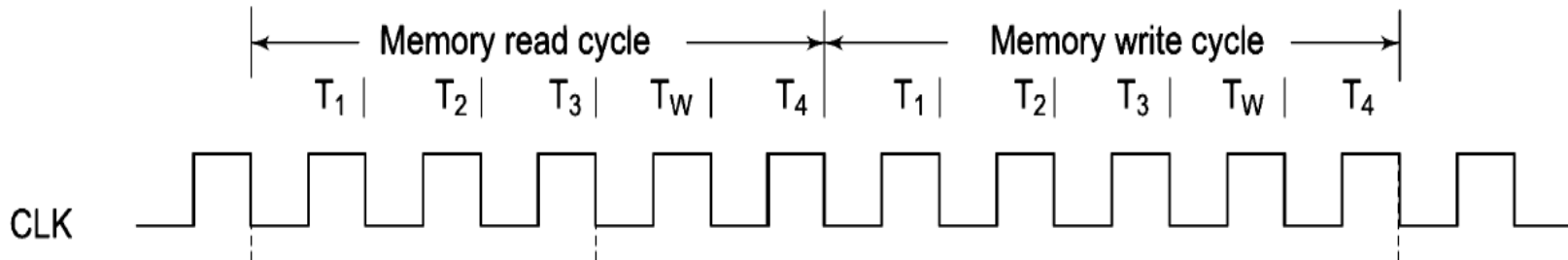
- ciclul de citire a instructiunii din memorie (faza de fetch – citirea codului instructiunii de executat din memoria de programe);
- ciclul de decodificare a instructiunii (de recunoastere de catre unitatea de comanda a instructiunii);
- ciclul de citire operanzi (daca instructiunea respectiva necesita operanzi);
- ciclul de executie propriu-zisa (se executa operatia ceruta de instructiune);
- ciclul de scriere a rezultatului.

Ciclii se executa in ritmul semnalelor de ceas aplicat procesorului (T – perioada elementara), numarul acestora depinzand de tipul ciclului si de procesor.

Exemplul 1



Exemplul 2



Reprezentarea instructiunilor

Fiecare instructiune este reprezentata binar in memorie pe 1 sau mai multi octeti, functie de complexitatea acesteia (la procesoarele Von Neumann - CISC).

De exemplu:

- Cod instructiune
- Operand 1
- Operand 2

Codul instructiunii este organizat pe campuri de biti cu diverse semnificatii care permit decodificarea instructiunii respective.

Pentru a inlesni scrierea programelor se folosesc mnemonici in locul instructiunilor in format binar (sau hexazecimal). Translatarea acestora in format binar se realizeaza cu programe de tip Asamblor.

Moduri de adresare

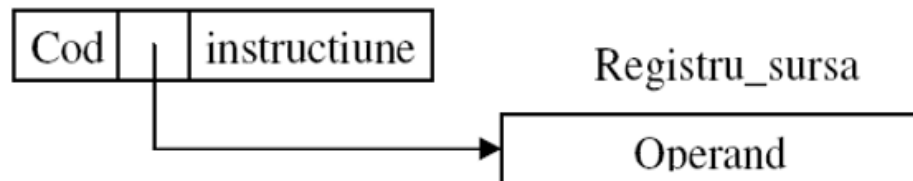
Modul de adresare este modalitatea (mecanismul) prin care, pe baza informatiilor continute de instructiunea curenta se poate calcula adresa efectiva pentru accesarea operandului (operandzilor), respectiv pentru stocarea rezultatului.

Set general de instructiuni, de lungime variabila, fiecare instructiune fiind reprezentata pe 1, 2, 3 sau 4 octeti.

1012h: 21h	LD HL, 1234h	Load
1013h: 34h		
1014h: 12h		
1015h: 3Eh	LD A, 45h	
1016h: 45h		
1017h: 2Ch	LD, B, A	
1018h: 3Ah	LD A, (HL)	
1019h:		

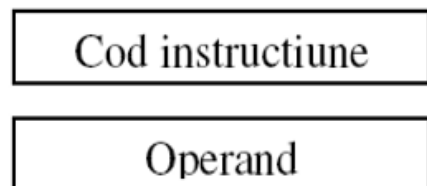
END C2 7 oct 2025

Adresarea la registru Un camp din codul instructiunii (in format binar) specifica un registru care contine operandul



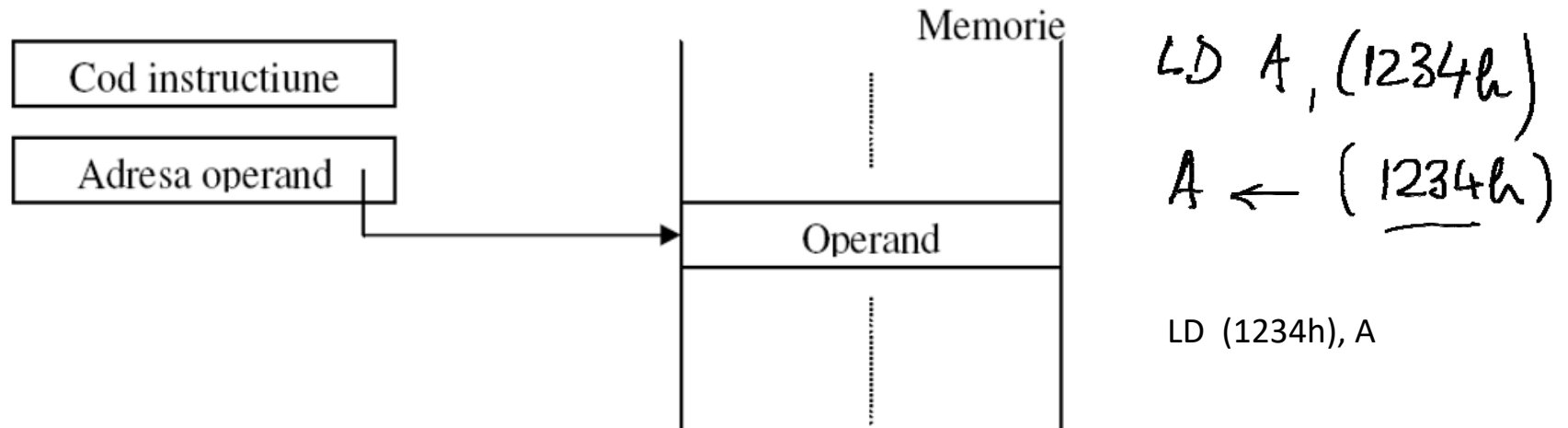
dest sursa 01 010 101 = 5th
 LD R2, R5
 R2 ← R5
 tip cod cod
 inst/r reg reg
 (Op) dest sursa
 transfer dest ← sursa
 transfer (move)

Adresarea imediata Operandul (sau operanzii) sunt specificati prin octet (octeti) care urmeaza dupa codul instructiunii (in format binar)

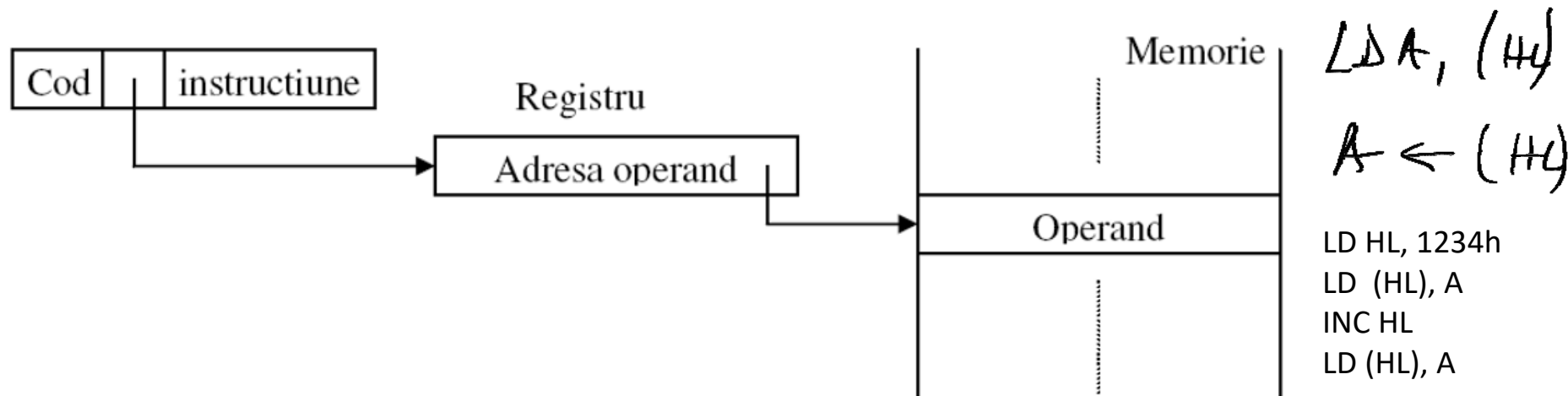


LD R3, 15h
 dest operand
 (valoare imediata)
 R3 ← 15h
 (load LD)

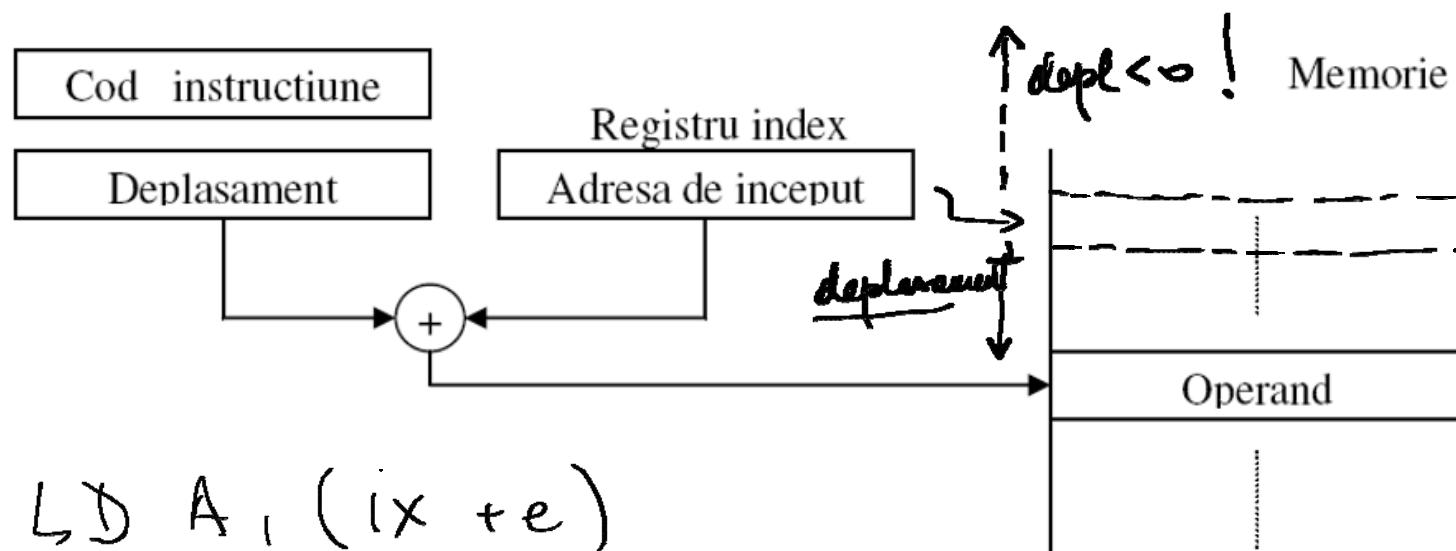
Adresarea directa Adresa efectiva a operandului este specificata in instructiune, dupa codul instructiunii.



Adresarea indirecta prin registru In instructiune se specifica un registru care contine adresa efectiva a operandului aflat in memorie.



Adresarea indexata Adresa efectiva a operandului se obtine din insumarea continutului unui registru index al procesorului cu un numar exprimat in CC2 (deplasament) care urmeaza dupa codul instructiunii.



$LD A, (ix + e)$

$A \leftarrow (ix + e)$

LD IX, 1000h
LD A, (IX + FEh)

FEh = 1111 1110 // 1111 1101 // 1 000 0010 = -2 CC2 !!!

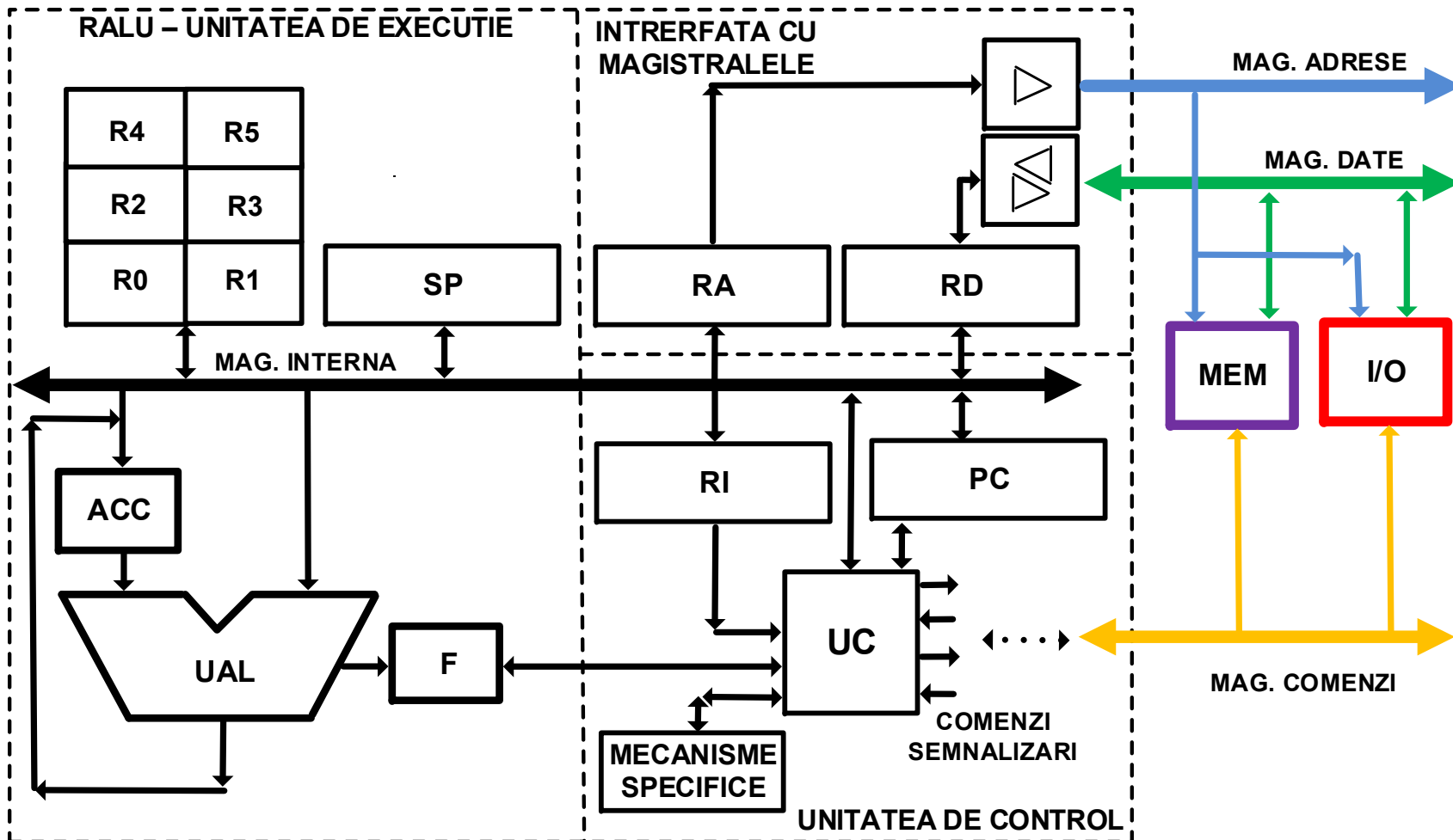
Exemple executii instructiuni

inc R0 realizeaza $R0 \leftarrow R0 + 1$; se reprezinta pe o singura locatie de memorie (cuvant / octet) care contine codul instructiunii (adresare la registru)

Executie interna in procesor:

Operatie	Comentariu
$RA \leftarrow PC$	Adresa din PC a instructiunii de executat se transfera in RA in vederea adresarii memoriei prin magistala de adrese
$RD \leftarrow \text{mem}(RA)$	Se preia in RD codul instructiunii de executat din memorie (fetch) prin magistrala de date
$RI \leftarrow RD$	Se transfera in RI codul instructiunii in vederea decodificarii acestuia; UC recunoaste instructiunea inc R0
$ACC \leftarrow R0$	UC incepe executia propriu-zisa prin trecerea lui R0 in ACC
$ACC \leftarrow ACC + 1$	Se incrementeaza ACC
$R0 \leftarrow ACC$	Se depune rezultatul in R0
$PC \leftarrow PC + 1$	Se actualizeaza PC (cu adresa urmatoarei instructiuni de executat), considerand ca inc R0 a ocupat o singura locatie de memorie

STRUCTURA UNUI PROCESOR VON NEUMANN



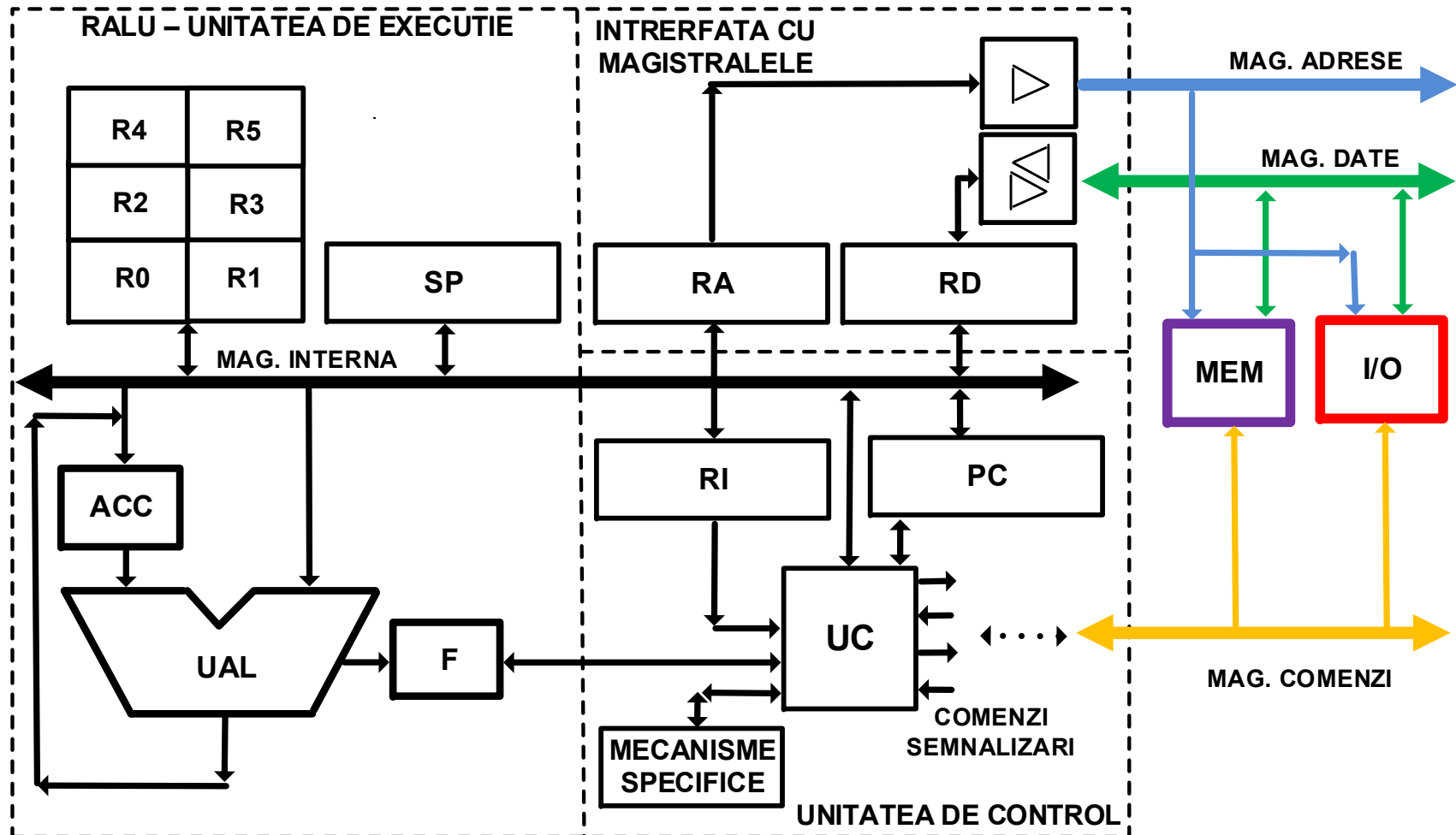
add Acc, operand, ; realizeaza $Acc \leftarrow Acc + operand$; se reprezinta pe 2 locatii de memorie (2 cuvante / 2 octeti) care contin codul instructiunii, respectiv operandul (adresare imediata)

Executie interna in procesor:

Operatie	Comentariu
$RA \leftarrow PC$	Adresa din PC a instructiunii de executat se transfera in RA in vederea adresarii memoriei prin magistala de adrese
$RD \leftarrow mem(RA)$	Se preia in RD codul instructiunii de executat din memorie (fetch) prin magistrala de date
$RI \leftarrow RD$	Se transfera in RI codul instructiunii in vederea decodificarii acestuia; UC recunoaste instructiunea add Acc, operand
$PC \leftarrow PC + 1$	Se actualizeaza PC pentru adresarea operandului
$RA \leftarrow PC$	Adresa din PC a operandului (adresa directa) se transfera in RA in vederea adresarii memoriei
$RD \leftarrow mem(RA)$	Se preia in RD operandul din memorie
$ACC \leftarrow ACC + RD$	Se executa operatia de adunare
$PC \leftarrow PC + 1$	Se actualizeaza PC (cu adresa urmatoarei instructiuni de executat)

END 14 oct 2025

STRUCTURA UNUI PROCESOR VON NEUMANN

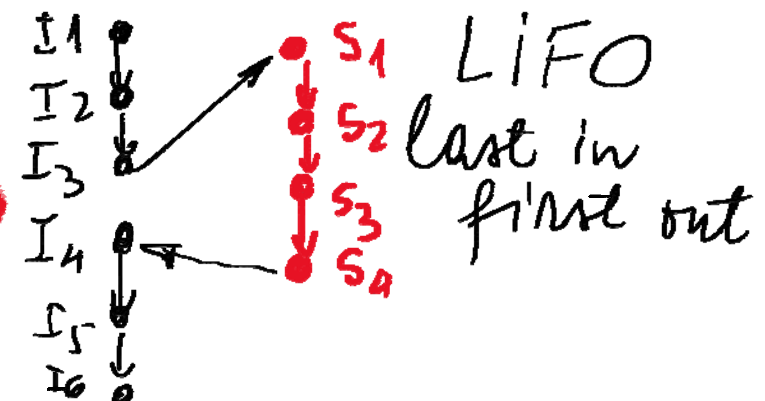
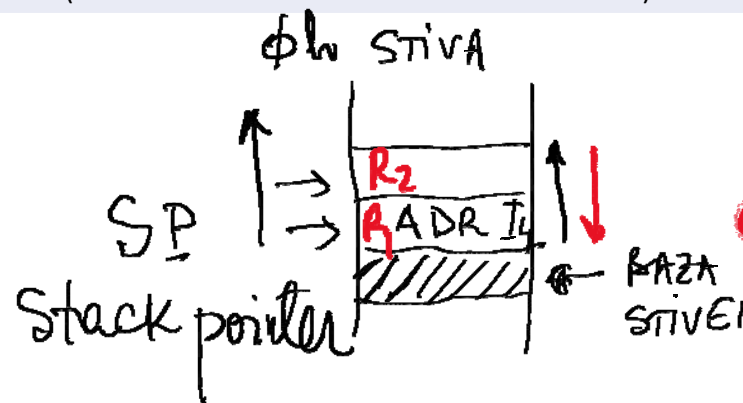


push R1 ; realizeaza depunerea in stiva a lui R1; se reprezinta pe o singura locatie de memorie (cuvant / octet) care contine codul instructiunii (adresare la registru)

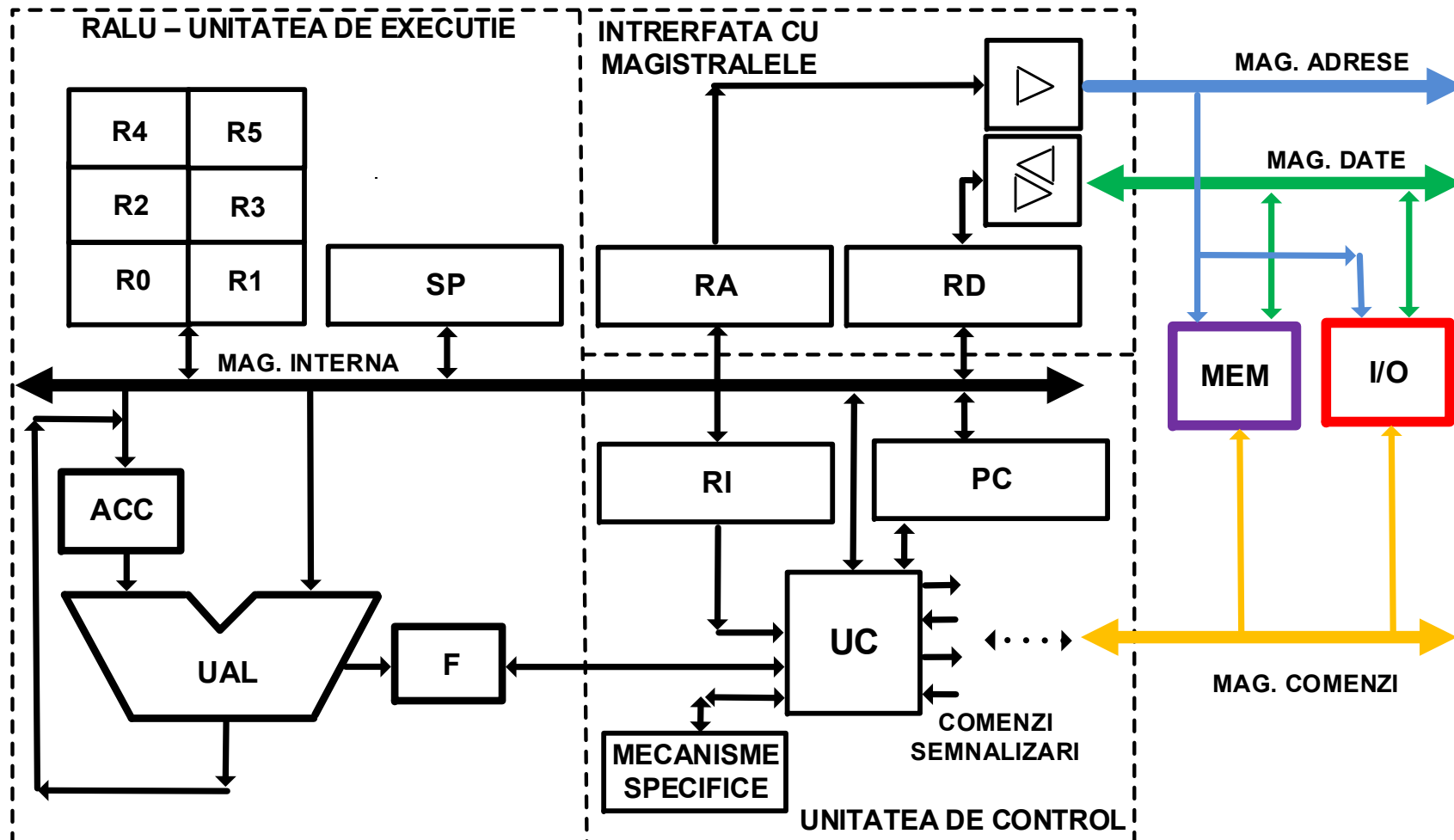
Executie interna in procesor:

Operatie	Comentariu
$RA \leftarrow PC$	Adresa din PC a instructiunii de executat se transfera in RA in vederea adresarii memoriei prin magistala de adrese
$RD \leftarrow \text{mem}(RA)$	Se preia in RD codul instructiunii de executat din memorie (fetch) prin magistrala de date
$RI \leftarrow RD$	Se transfera in RI codul instructiunii in vederea decodificarii acestuia; UC recunoaste instructiunea push R1
$SP \leftarrow SP-1$	Se formeaza adresa varfului stivei unde va fi depus R0 prin decrementarea lui SP
$RA \leftarrow SP$	Se depune adresa varfului stivei in RA in vederea adresarii stivei
$RD \leftarrow R1$	Continutul lui R1 se depune in RD in vederea salvarii (scrierii) in stiva
$\text{mem}(RA) \leftarrow RD$	Se salveaza (scrie) in varful stivei informatia din RD
$PC \leftarrow PC+1$	Se actualizeaza PC (cu adresa urmatoarei instructiuni de executat)

PUSH R1
 PUSH R2
 ...
 POP R2 R1
 POP R1 R2



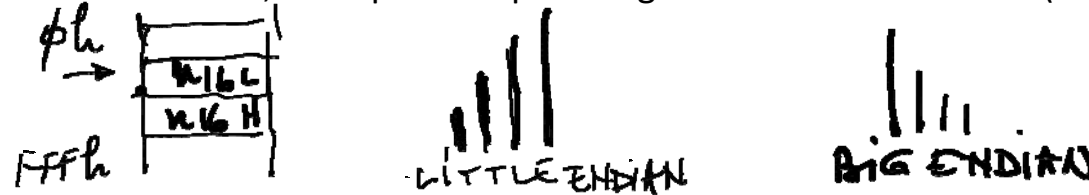
STRUCTURA UNUI PROCESOR VON NEUMANN



$LD(HL), n_{16}$ $n_{16}H$
 $n_{16}L$

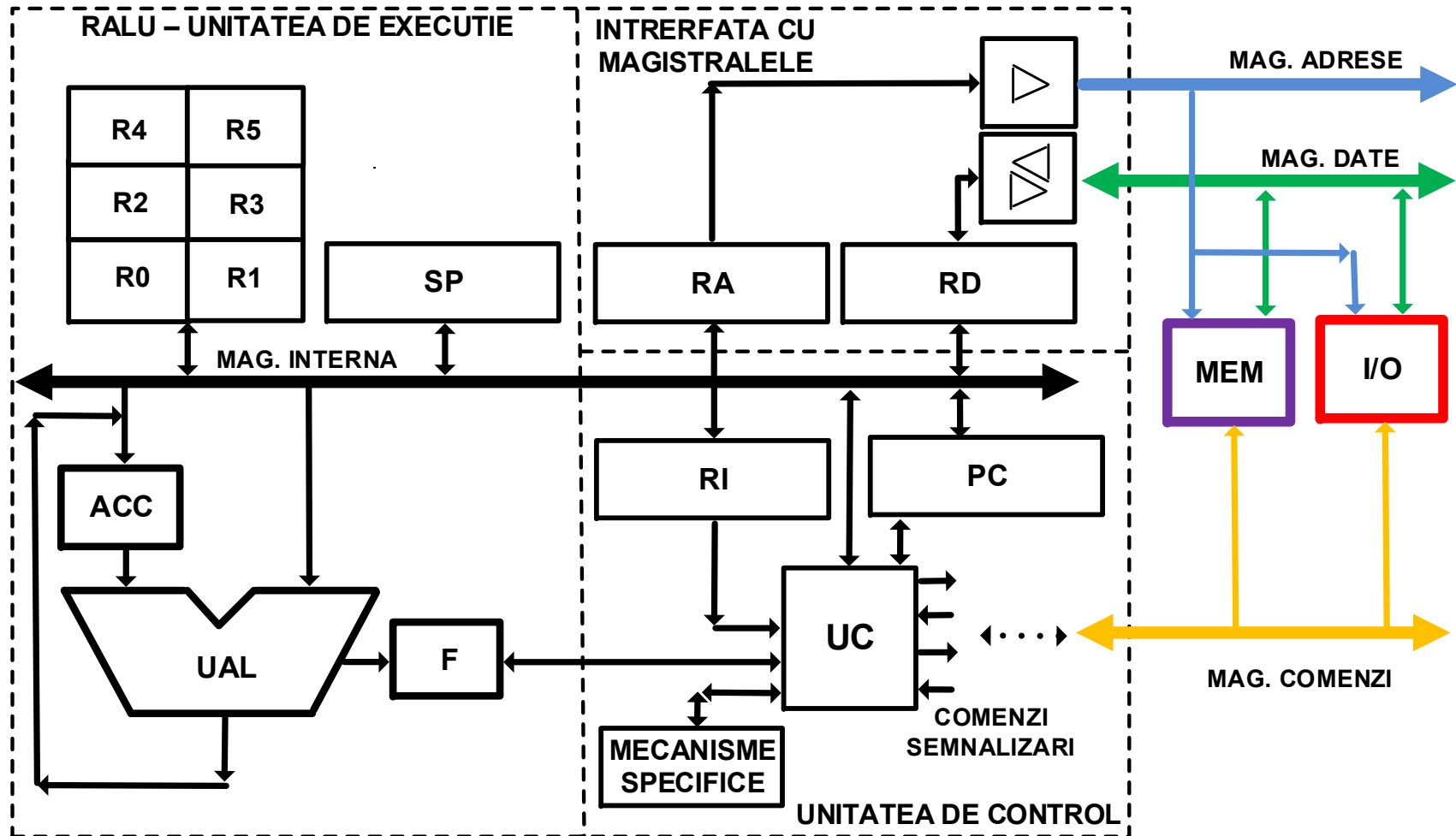
pop R2 ; realizeaza restaurarea din stiva a lui R2; se reprezinta pe o singura locatie de memorie (cuvant / octet) care contine codul instructiunii

Executie interna in procesor:



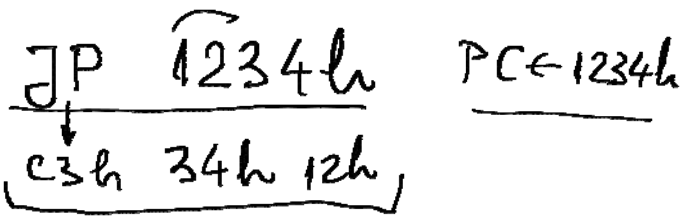
Operatie	Comentariu
$RA \leftarrow PC$	Adresa din PC a instructiunii de executat se transfera in RA in vederea adresarii memoriei prin magistala de adrese
$RD \leftarrow mem(RA)$	Se preia in RD codul instructiunii de executat din memorie (fetch) prin magistrala de date
$RI \leftarrow RD$	Se transfera in RI codul instructiunii in vederea decodificarii acestuia; UC recunoaste instructiunea pop R2
$RA \leftarrow SP$	Se depune adresa varfului stivei in RA in vederea adresarii stivei
$RD \leftarrow mem(RA)$	Continutul din varful stivei (adresa aflandu-se in RA) se depune in RD
$R2 \leftarrow RD$	Se depune in R2 informatia preluata din varful stivei
$SP \leftarrow SP+1$	Se formeaza adresa noului varf al stivei
$PC \leftarrow PC+1$	Se actualizeaza PC (cu adresa urmatoarei instructiuni de executat)

STRUCTURA UNUI PROCESOR VON NEUMANN



jmp adresa ; realizeaza un salt neconditionat **PC←adresa**; se reprezinta pe 2 locatii de memorie (cuvant / octet) care contin codul instructiunii, respectiv adresa de memorie unde se face saltul

Executie interna in procesor:



Operatie	Comentariu
RA ← PC	Adresa din PC a instructiunii de executat se transfera in RA in vederea adresarii memoriei prin magistala de adrese
RD ← mem(RA)	Se preia in RD codul instructiunii de executat din memorie (fetch) prin magistrala de date
RI ← RD	Se transfera in RI codul instructiunii in vederea decodificarii acestuia; UC recunoaste instructiunea jmp adresa
PC ← PC+1	Se actualizeaza PC pentru preluarea operandului (adresei de salt)
RA ← PC	Se incarca adresa operandului (adresa de salt) in RA
RD ← mem(RA)	Se preia in RD adresa de salt
PC ← RD	Se depune in CP adresa de salt de unde va fi preluata urmatoarea instructiune de executat

①

[C3h]

②

[34h]

[12h] ③

PC 1234h ④

jz adresa ; realizeaza un salt conditionat **PC←adresa** daca indicatorul Z=1, altfel trece la instructiunea urmatoare; se reprezinta pe 2 locatii de memorie (cuvant / octet) care contin codul instructiunii, respectiv adresa de memorie unde se face saltul conditionat

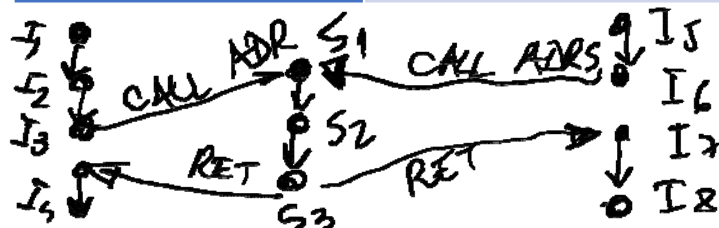
Executie interna in procesor:

Operatie	Comentariu
RA ← PC	Adresa din PC a instructiunii de executat se transfera in RA in vederea adresarii memoriei prin magistala de adrese
RD ← mem(RA)	Se preia in RD codul instructiunii de executat din memorie (fetch) prin magistrala de date
RI ← RD	Se transfera in RI codul instructiunii in vederea decodificarii acestuia; UC recunoaste instructiunea jz adresa
PC ← PC+1	Se actualizeaza PC pentru preluarea operandului (adresei de salt)
RA ← PC	Se incarca adresa operandului (adresa de salt) in RA
RD ← mem(RA)	Se preia in RD adresa de salt
Z = 1 ?	Daca da, PC ← RD (se incarca adresa de salt)
	Daca nu, PC ← PC+1 (se trece la adresa urmatoarei instructiuni de dupa jz adresa)
	Cu noul continut al PC se trece la adresa unde va fi preluata urmatoarea instructiune de executat

call adresa ; realizeaza apelul neconditionat al unei proceduri (subrutina) aflata la adresa **adresa**; se reprezinta pe 2 locatii de memorie (cuvant / octet) care contin codul instructiunii, respectiv adresa procedurii (adresa de la care este scrisa procedura)

Executie interna in procesor:

Operatie	Comentariu
$RA \leftarrow PC$	Adresa din PC a instructiunii de executat se transfera in RA in vederea adresarii memoriei prin magistala de adrese
$RD \leftarrow \text{mem}(RA)$	Se preia in RD codul instructiunii de executat din memorie (fetch) prin magistrala de date
$RI \leftarrow RD$	Se transfera in RI codul instructiunii in vederea decodificarii acestuia; UC recunoaste instructiunea call adresa
$PC \leftarrow PC+1$	Se actualizeaza PC; in PC este dupa incrementare adresa procedurii; OBS. Nu se va putea prelua acum adresa deoarece nu exista loc pentru salvarea acesteia in UC!
$PC \leftarrow PC+1$	Se formeaza in PC adresa urmatoarei instructiuni de executat de dupa revenirea din procedura
$SP \leftarrow SP-1$	Se pregateste locatia de stiva (varful stivei) in care va fi salvat continutul PC actualizat
$RA \leftarrow SP$	Se pregateste adresa varfului stivei pentru transmitere pe magistrala de adrese
$RD \leftarrow PC$	Se pregatesc datele care vor fi depuse in varful stivei prin magistrala de date
$\text{mem}(RA) \leftarrow RD$	Se depune in varful stivei adresa de unde va fi preluata urmatoarea instructiune de executat la revenirea din procedura
$PC \leftarrow PC-1$	Se revine in PC la adresa de memorie de unde va fi preluata adresa procedurii
$RA \leftarrow PC$	Se incarca adresa operandului (adresa procedurii) in RA
$RD \leftarrow \text{mem}(RA)$	Se preia in RD operandul (adresa procedurii)
$PC \leftarrow RD$	Se transfera in RD adresa procedurii de la care procesorul va continua cu urmatoarea instructiune de executat



call adresa ; realizeaza apelul neconditionat al unei proceduri (subrutina) aflata la adresa **adresa**; se reprezinta pe 2 locatii de memorie (cuvant / octet) care contin codul instructiunii, respectiv adresa procedurii (adresa de la care este scrisa procedura)

Executie interna in procesor:

Operatie	Comentariu
$RA \leftarrow PC$	Adresa din PC a instructiunii de executat se transfera in RA in vederea adresarii memoriei prin magistala de adrese
$RD \leftarrow \text{mem}(RA)$	Se preia in RD codul instructiunii de executat din memorie (fetch) prin magistrala de date
$RI \leftarrow RD$	Se transfera in RI codul instructiunii in vederea decodificarii acestuia; UC recunoaste instructiunea call adresa
$PC \leftarrow PC+1$	Se actualizeaza PC; in PC este dupa incrementare adresa procedurii; OBS. Nu se va putea prelua acum adresa deoarece nu exista loc pentru salvarea acesteia in UC!
$PC \leftarrow PC+1$	Se formeaza in PC adresa urmatoarei instructiuni de executat de dupa revenirea din procedura
$SP \leftarrow SP-1$	Se pregateste locatia de stiva (varful stivei) in care va fi salvat continutul PC actualizat
$RA \leftarrow SP$	Se pregateste adresa varfului stivei pentru transmitere pe magistrala de adrese
$RD \leftarrow PC$	Se pregatesc datele care vor fi depuse in varful stivei prin magistrala de date
$\text{mem}(RA) \leftarrow RD$	Se depune in varful stivei adresa de unde va fi preluata urmatoarea instructiune de executat la revenirea din procedura
$PC \leftarrow PC-1$	Se revine in PC la adresa de memorie de unde va fi preluata adresa procedurii
$RA \leftarrow PC$	Se incarca adresa operandului (adresa procedurii) in RA
$RD \leftarrow \text{mem}(RA)$	Se preia in RD operandul (adresa procedurii)
$PC \leftarrow RD$	Se transfera in RD adresa procedurii de la care procesorul va continua cu urmatoarea instructiune de executat

ret ; realizeaza revenirea dintr-o procedura (subrutina); se reprezinta pe o locatie de memorie (cuvant / octet) care contine codul instructiunii

END 21 oct 2015

Executie interna in procesor:

Operatie	Comentariu
$RA \leftarrow PC$	Adresa din PC a instructiunii de executat se transfera in RA in vederea adresarii memoriei prin magistala de adrese
$RD \leftarrow \text{mem}(RA)$	Se preia in RD codul instructiunii de executat din memorie (fetch) prin magistala de date prin magistrala de date
$RI \leftarrow RD$	Se transfera in RI codul instructiunii in vederea decodificarii acestuia; UC recunoaste instructiunea ret
$RA \leftarrow SP$	Se pregateste adresa varfului stivei pentru transmitere pe magistrala de adrese in vederea preluarii (extragerii) adresei de revenire salvata anterior
$RD \leftarrow \text{mem}(RA)$	Se preia adresa de revenire din stiva
$SP \leftarrow SP+1$	Se reface varful stivei dupa extragerea efectuata
$PC \leftarrow RD$	Se transfera in PC adresa urmatoarei instructiuni de executat de dupa procedura